

CALCJUD

Ferramenta de Cálculos Judiciais de IRPF

Manual de Banco de Dados — Acesso, Backup e Manutenção

Tribunal Regional Federal da 2ª Região (TRF2)

30/08/2026

Sumário

- [1. Visão geral do banco de dados](#)
- [2. Como acessar o banco de dados](#)
- [3. Backup de todos os dados do sistema](#)
- [4. Manutenção e melhoria contínua](#)
- [5. Referência rápida — tabelas do sistema](#)

Visão geral do banco de dados

O CALCJUD utiliza um banco de dados PostgreSQL hospedado e totalmente gerenciado pelo Supabase (Backend-as-a-Service), que também fornece autenticação de usuários, API REST/RPC automática (PostgREST) e controle de acesso por linha (Row Level Security — RLS).

- Projeto Supabase: identificador (Project Ref) xitpsqtcxraejlxvnm — URL <https://xitpsqtcxraejlxvnm.supabase.co>.
- O esquema do banco é versionado no repositório de código do sistema: `supabase/schema.sql` reúne o esquema completo (script idempotente, pode ser reaplicado com segurança) e `supabase/migrations/` guarda as alterações incrementais já aplicadas ao longo do tempo.
- Toda a lógica de cálculo do IRPF roda no navegador (front-end); o banco de dados funciona apenas como fonte dos parâmetros/tabelas auxiliares usados no cálculo e como repositório do histórico de cálculos já realizados.
- Segurança: RLS habilitada em todas as tabelas de negócio; leitura pública liberada apenas para as tabelas fiscais/paramétricas e para os cálculos já realizados; qualquer escrita (inclusão, edição ou exclusão) exige usuário administrador autenticado.

Como acessar o banco de dados

2.1 Acesso pelo painel do Supabase (Dashboard) — recomendado

1. Acesse <https://supabase.com/dashboard/organizations> e faça login com uma conta que já tenha sido convidada para a organização/projeto do CALCJUD.
2. Se o login não mostrar nenhuma organização, a conta ainda não foi convidada — peça a um administrador atual do projeto para adicioná-la em Project Settings → Team.
3. Selecione o projeto de Project Ref xitpsqtcxraejlxvnm, ou acesse diretamente <https://supabase.com/dashboard/project/xitpsqtcxraejlxvnm>.
4. No menu lateral, as áreas mais usadas são: Table Editor (ver e editar dados linha a linha), SQL Editor (executar consultas e scripts SQL), Authentication (gerenciar contas de login), Database → Tables / Policies (estrutura das tabelas e regras de segurança), Database → Backups (cópias de segurança), Project Settings → API (URL e chaves de acesso) e Project Settings → Database (string de conexão para acesso direto).

2.2 Acesso direto via linha de comando (psql)

5. No painel, vá em Project Settings → Database e copie a Connection string. Há duas opções: conexão direta (porta 5432, recomendada para tarefas administrativas e migrações) e conexão via pooler (porta 6543, modo transaction, recomendada para aplicações com muitas conexões simultâneas).
6. Com um cliente PostgreSQL instalado (ex.: psql), conecte com:

```
psql "postgres://postgres:[SUA-SENHA]@db.xitpsqtcxraejlxvnm.supabase.co:5432/postgres"
```

7. A senha do usuário postgres do banco é definida/redefinida em Project Settings → Database → Reset database password — é diferente das senhas de login dos administradores do CALCJUD dentro da aplicação.

2.3 Acesso via Supabase CLI (para desenvolvimento e scripts)

8. Instale a CLI: `npm install -g supabase` (ou execute avulso com `npx supabase`).
9. Autentique-se: `supabase login`.

10. Vincule o projeto local ao projeto remoto: `supabase link --project-ref xitpsqtcxraejzlxvnmn`.
11. Comandos úteis no dia a dia: `supabase db pull` (traz o esquema remoto para o repositório local), `supabase db push` (aplica as migrações locais no banco remoto) e `supabase db diff` (gera um novo arquivo de migração a partir de alterações feitas manualmente pelo painel).

2.4 Acesso pela própria aplicação (uso comum, somente leitura pública)

A aplicação usa a URL e a chave anon/public do Supabase (variáveis `VITE_SUPABASE_URL` e `VITE_SUPABASE_PUBLISHABLE_KEY`, configuradas a partir de `.env.example` na raiz do repositório). Essa chave não concede acesso administrativo — todo acesso de escrita continua sujeito a login e às políticas de RLS descritas na seção 1.

Backup de todos os dados do sistema

3.1 Backups automáticos do Supabase

O Supabase mantém backups automáticos do banco; a frequência e o prazo de retenção dependem do plano contratado para o projeto (planos pagos costumam oferecer backups diários com maior retenção e, opcionalmente, Point-in-Time Recovery — restauração para qualquer instante dentro do período de retenção).

12. No painel, acesse Database → Backups para ver quais backups automáticos existem e qual é o plano/retenção vigente.
13. Para restaurar um backup, use a opção Restore ao lado do backup desejado. A operação substitui os dados atuais do projeto pelos do backup escolhido — é uma operação de alto impacto e deve ser confirmada com toda a equipe antes de ser executada.
14. Confirme periodicamente, junto ao responsável pela conta Supabase da organização, qual é o plano contratado e a política de retenção — sem essa confirmação não é possível garantir a real capacidade de recuperação em caso de incidente.

3.2 Backup manual (antes de qualquer manutenção arriscada)

Além dos backups automáticos do Supabase, é recomendado gerar uma cópia manual completa antes de aplicar uma migração de esquema ou qualquer alteração maior de dados, usando `pg_dump` — ferramenta padrão de backup do PostgreSQL:

```
pg_dump "postgresql://postgres:[SUA-SENHA]@db.xitpsqtcxraejzlxvnmn.supabase.co:5432/postgres" -F c -f calcjud_backup_AAAAMMDD.dump
```

O parâmetro `-F c` gera um arquivo em formato comprimido/binário do `pg_dump` (recomendado para restauração); para gerar um script SQL legível em texto puro, use `-F p -f calcjud_backup_AAAAMMDD.sql` no lugar.

Para restaurar um backup gerado dessa forma:

```
pg_restore --clean --if-exists -d "postgresql://postgres:[SUA-SENHA]@db.xitpsqtcxraejzlxvnmn.supabase.co:5432/postgres" calcjud_backup_AAAAMMDD.dump
```

As opções `--clean --if-exists` fazem a restauração remover as versões atuais dos objetos antes de recriá-los, evitando conflitos com dados já existentes no banco de destino.

3.3 Exportação pontual de uma tabela (CSV)

Para conferências ou auditorias pontuais de uma única tabela (por exemplo, calculos, antes de uma limpeza de dados antigos), o Table Editor do painel permite exportar o conteúdo filtrado da tabela diretamente em CSV, sem precisar de linha de comando.

3.4 Frequência recomendada e cuidados com os dados

- Gerar um backup manual sempre antes de aplicar uma migração de esquema, uma limpeza/exclusão em massa de dados ou qualquer alteração administrativa de maior risco.
- Manter também backups periódicos (por exemplo, semanais) guardados fora da própria conta Supabase — em outro provedor de nuvem ou em armazenamento local seguro — como camada extra de proteção, independente da conta do Supabase.
- Os dados das tabelas calculos e admin_users podem conter dados pessoais (nome do autor do processo, número do processo, e-mails de administradores); tratar os arquivos de backup com o mesmo cuidado de confidencialidade e retenção exigido pela LGPD para esse tipo de informação.
- O próprio esquema do banco (supabase/schema.sql e supabase/migrations/) já funciona como um "backup" da estrutura (tabelas, funções, triggers, políticas). Qualquer alteração estrutural feita diretamente pelo painel deve ser replicada nesses arquivos (ou gerada via supabase db diff) e enviada ao repositório, para não haver divergência entre o banco em produção e o que está versionado no código.

Manutenção e melhoria contínua

- Segurança — trocar a senha do administrador inicial de bootstrap: o script supabase/schema.sql contém, em texto claro, e-mail e senha de um administrador criado no processo de instalação do banco; essa senha deve ser rotacionada assim que possível pelo painel (Authentication → Users) e, se possível, o segredo removido do histórico do repositório.
- Atualização anual dos parâmetros fiscais: as tabelas ir_parametros, ir_faixas, salario_minimo e indices_economicos/taxas_historicas (SELIC, poupança etc.) precisam ser atualizadas a cada início de ano-calendário com os novos valores oficiais; hoje isso é feito manualmente pelo Table Editor ou SQL Editor — vale planejar um processo/checklist recorrente para que o cálculo do CALCJUD nunca fique defasado em relação aos índices oficiais vigentes.
- Revisão periódica da lista de administradores (admin_users) e das políticas de RLS: conferir de tempos em tempos se todos os e-mails com privilégio de administrador ainda são de pessoas que devem ter esse acesso, removendo quem não precisa mais.
- Acompanhar o Database → Advisors do painel do Supabase: ele aponta automaticamente alertas de segurança (por exemplo, uma tabela nova sem RLS habilitada) e de performance (por exemplo, um índice faltando) — vale revisar periodicamente, especialmente logo após qualquer alteração de esquema.
- Acompanhar Logs/Reports do painel para identificar erros recorrentes de API ou consultas lentas antes que virem um problema perceptível para quem usa o sistema.
- Monitorar o uso do projeto (tamanho do banco, número de requisições, armazenamento) em Project Settings → Usage/Billing, para antecipar a necessidade de upgrade de plano antes de atingir algum limite.
- Antes de publicar qualquer mudança de esquema ou de regra de cálculo, rodar a suíte de testes do projeto (bun run test para os testes unitários das regras de cálculo, e os testes end-to-end via Playwright) e, sempre que a mudança envolver o banco, gerar um backup manual antes (ver seção 3.2).
- Manter as migrações incrementais (supabase/migrations/) como histórico definitivo de mudanças de esquema: toda alteração estrutural deve virar um novo arquivo de migração com timestamp, nunca uma edição retroativa de uma migração já aplicada em produção.

- Manter as dependências do projeto (package.json) razoavelmente atualizadas — em especial a biblioteca oficial @supabase/supabase-js — para se beneficiar de correções de segurança e de compatibilidade com o Supabase.

Referência rápida — tabelas do sistema

Tabela consolidada de todas as tabelas do banco de dados do CALCJUD e sua finalidade — útil como referência rápida ao navegar pelo Table Editor ou pelo SQL Editor do Supabase.

Tabela	Finalidade
ir_parametros	Parâmetros gerais de IR por ano-calendário: teto do desconto simplificado e data de início de correção do ano.
ir_faixas	Faixas progressivas de alíquota e parcela a deduzir, por ano-calendário.
calculos	Histórico de cálculos realizados (tipo — ajuste anual ou retificação —, ano, processo, autor, dados de entrada e resultado em JSONB); é a tabela usada na Consulta pública por ID.
salario_minimo	Série histórica de salário mínimo por data de referência, usada para apurar o teto do RPV (60 salários mínimos).
indices_economicos	Cadastro de índices de correção monetária e de juros (ex.: SELIC, poupança, UFIR), com sua natureza (CORRECAO ou JUROS).
taxas_historicas	Série mensal de percentuais por índice, com fator multiplicador e fator acumulado recalculados automaticamente (trigger handle_taxas_historicas_write / função recalculate_taxas_historicas) sempre que uma taxa é inserida, alterada ou removida.
templates_calculo	Templates de cálculo correspondentes aos tipos de correção disponíveis na Retificação (SELIC, SELIC + poupança, sem correção).
regras_subperiodo	Regras de correção/juros vigentes por sub-período dentro de cada template (qual índice de correção e de juros usar em cada intervalo de datas).
admin_invites	Convites pendentes/aceitos de novos administradores, por e-mail.
admin_users	Usuários com privilégio de administrador (vinculados a auth.users do Supabase Auth).
system_settings	Chave única (linha singleton) com as chaves de disponibilidade pública: sistema inteiro, Ajuste Anual e Retificação.